MCNN on forward-looking sonar: reimplementation, a variable-size CNN, and how much of 99.9% is leakage

Report generated 2026-11-02 from `results/*.json` in the sonar-cnn repository.

1. Summary

- **The paper's number reproduces.** MCNN as described in Table 2 of Çimen & Bolat reaches 99.86% on a random 90/10 split, matching the 99.87% stated in the paper's text (the 99.98% in the abstract appears to be a typo).
- **Crop size alone nearly solves the task.** A random forest given only each image's height and width (no pixels) gets 98.17 ± 0.70%. Padding to 305x305 exposes that size to the network: MCNN trained on silhouettes with all sonar content removed still gets 98.3%.
- **Random splits are dominated by near-duplicate frames.** A 1-nearest-neighbour lookup on 32x32 thumbnails gets 100.00% on a random split; 71% of test images have a training image with correlation above 0.9.
- **Honest numbers are lower.** On a split by pseudo-sequence groups, MCNN drops to 97.2% with padding and 90.4% once size information is removed. The proposed VarCNN (fully convolutional, 185k parameters, no padding effect) holds at 96.6%.

2. Dataset

Valdenegro-Toro marine debris forward-looking sonar dataset (ARIS Explorer 3000, water tank), folder `md_fls_dataset/data/watertank-cropped` of `mvaldenegro/marine-debris-fls-datasets` at commit 4a77998. 2,364 PNG images in 10 classes; per-class counts match the paper's Table 1 exactly. Images are stored as RGBA with R=G=B and constant alpha, so one channel is used. Each image is a hand-cropped box around one object, and the box size depends strongly on the class (Table 1, Figure 1). File numbers do not follow recording order: consecutive file numbers are no more similar than random pairs of the same class (median thumbnail correlation 0.46 vs 0.45).

Table 1. Images per class and crop size.

Class	Images	Mean h x w (px)	Height range	Width range
bottle	449	88 x 83	57-98	72-103
can	367	75 x 70	55-96	47-105
chain	226	90 x 70	50-235	48-148
drink-carton	349	57 x 59	42-74	43-88
hook	133	109 x 102	66-173	89-115
propeller	137	119 x 123	80-135	96-142
shampoo-bottle	99	69 x 200	67-75	173-266
standing-bottle	65	46 x 173	42-50	161-181
tire	331	98 x 96	55-143	73-148
valve	208	99 x 87	76-121	56-102
total	**2364**		**42-235**	**43-266**

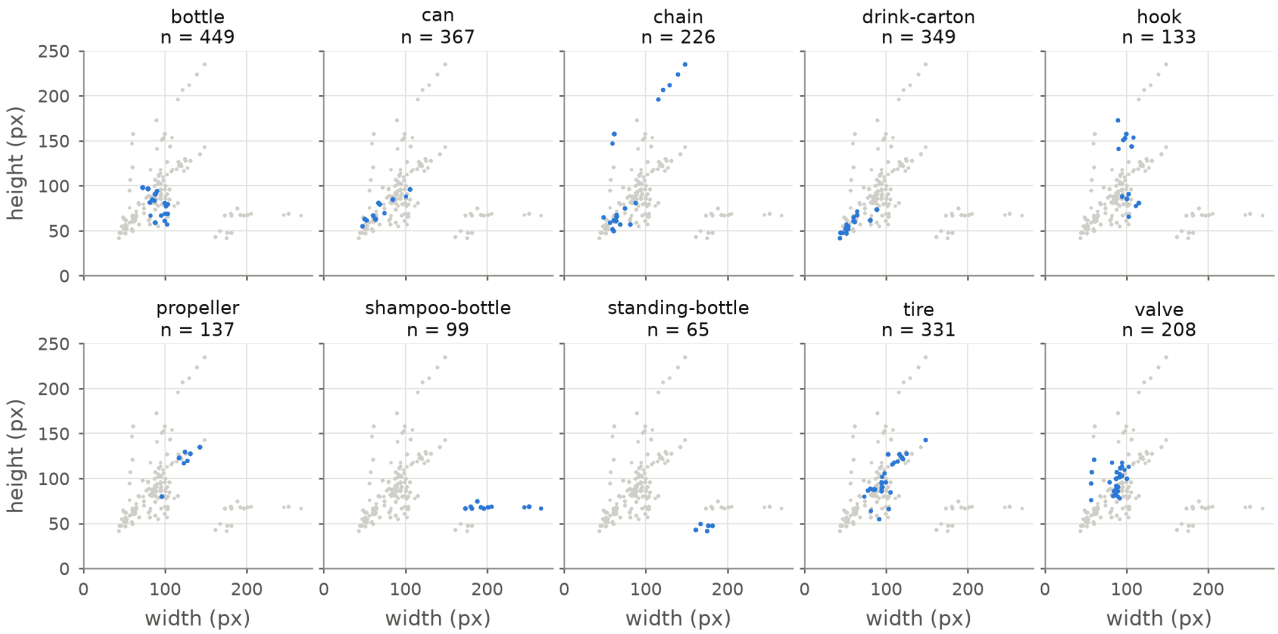


Figure 1. Crop height vs width for each class (blue) against all images (grey). Standing and shampoo bottles are long and flat, drink cartons small and square, propellers and hooks large: size and aspect ratio alone separate most classes.

3. Architectures

3.1 MCNN (paper replication)

Built from Table 2 of the paper for 305x305 zero-padded input. 2,797,706 parameters, of which 2,654,464 (95%) are in fc1. Two departures from the text, both because the paper is ambiguous: a ReLU after every convolution (the text mentions only one, after conv6), and 'same' padding on the 2x2 conv6 so the output is the paper's 9x9x128. conv1-3 use padding 2: the paper lists padding 1 for conv2 and conv3, which would give 76 and 38 instead of its stated 77 and 39.

Table 2. MCNN layers (input 1 x 305 x 305).

Layer	Config (k, ch, stride, pad)	Output (C x H x W)	Params
conv1 + ReLU	5x5, 32, s2, p2	32 x 153 x 153	832
conv2 + ReLU	5x5, 32, s2, p2	32 x 77 x 77	25,632
conv3 + ReLU	5x5, 32, s2, p2	32 x 39 x 39	25,632
MaxPool	2x2, s2	32 x 19 x 19	0
conv4 + ReLU	3x3, 64, s1, p1	64 x 19 x 19	18,496
conv5 + ReLU	3x3, 64, s1, p1	64 x 19 x 19	36,928
MaxPool	2x2, s2	64 x 9 x 9	0
conv6	2x2, 128, s1, psame	128 x 9 x 9	32,896
BatchNorm + ReLU	128 ch	128 x 9 x 9	256
flatten		10,368	0
fc1 + ReLU	10,368 -> 256	256	2,654,464
fc2 (softmax)	256 -> 10	10	2,570
total			**2,797,706**

3.2 VarCNN (variable-size input)

A fully convolutional variant that takes each image at its original size. Batches are padded to the largest image only so they can be stacked into one tensor. After every layer the padded area is set back to zero (including after max-pooling, where an odd size would otherwise leave a real value in the padded area), BatchNorm statistics are computed over real pixels only, and the final pooling (global mean and global max, concatenated) uses real pixels only. In eval mode each sample's output is therefore identical to running that image alone; `tests/test_varcnn.py` checks this to 1e-5. Compared with MCNN, conv2 has stride 1 so that the smallest images (42 px) still have spatial extent at the end, and the 2.65M-parameter fc1 is replaced by global pooling: 184,650 parameters in total, 15x fewer than MCNN.

Table 3. VarCNN layers, with output sizes for the smallest and largest images in the dataset.

Layer	Config (k, ch, stride, pad)	42 x 43 in	235 x 266 in	Params
conv1 + masked BN + ReLU	5x5, 32, s2, p2	21x22	118x133	896
conv2 + masked BN + ReLU	5x5, 32, s1, p2	21x22	118x133	25,696
conv3 + masked BN + ReLU	5x5, 32, s2, p2	11x11	59x67	25,696
MaxPool + re-mask	2x2, s2	5x5	29x33	0
conv4 + masked BN + ReLU	3x3, 64, s1, p1	5x5	29x33	18,624
conv5 + masked BN + ReLU	3x3, 64, s1, p1	5x5	29x33	37,056
MaxPool + re-mask	2x2, s2	2x2	14x16	0
conv6 + masked BN + ReLU	3x3, 128, s1, p1	2x2	14x16	74,112
masked global mean + max	over real pixels	256	256	0
dropout 0.3, fc	256 -> 10	10	10	2,570
total				**184,650**

4. Training and evaluation

Table 4. Training configuration (identical for all experiments).

Setting	Value
Optimizer	AdamW, lr 1e-3, weight decay 1e-4, cosine decay to 0 over all steps
Batch size / epochs	32 / 30 (fixed-size inputs), 50 (randsize inputs)
Loss	cross-entropy, no class weighting
Augmentation	none, except the random resize of randsize inputs
Split	stratified 80/20 (one run at 90/10); seeds 0, 1, 2
Model selection	none: no validation set, accuracy at the final epoch (as in the paper)
Hardware	NVIDIA RTX 3050 Laptop (4 GB), PyTorch 2.15.0 + CUDA 13.2, Python 3.12

Table 5. Input modes. 'Probe' modes replace all sonar content with 1s, so the model can only use the crop's size and shape.

Input	What the model sees
pad	image centered on a 305x305 zero canvas (the paper's setup)
silhouette (probe)	as pad, content replaced by 1s
native	image at its original size (VarCNN only)
native-constant (probe)	as native, content replaced by 1s
randsize	image resized to random h, w, each uniform in [48, 160] and independent of class; new size every epoch for training, one fixed seeded size per test image
randsize-pad	as randsize, then centered on a 305x305 zero canvas (for MCNN)
...-constant (probe)	as above, content replaced by 1s

Splits. *Random*: stratified by class, the paper's protocol. *Group*: images are linked when their 32x32 mean-centered thumbnails have correlation 0.85 or more, and connected chains (single linkage) within a class form a group, an approximation of a stretch of one recording. 358 groups result. Whole groups are assigned to the test set in random order until it holds about 20% of each class, so no test image has a training image above 0.85 similarity.

5. Results

Table 6. All experiments: mean ± std over 3 seeds. 'Errors per seed' counts misclassified test images.

Model	Input	Split	Accuracy (%)	Macro-F1 (%)	Errors per seed
MCNN	pad (paper setup)	random	99.72 ± 0.26	99.79 ± 0.21	1/0/3 of 473
MCNN	pad, 90/10 split	random	99.86 ± 0.20	99.91 ± 0.13	1/0/0 of 237
MCNN	silhouette (no content)	random	98.31 ± 0.35	98.46 ± 0.32	10/6/8 of 473
MCNN	randsize-pad	random	99.44 ± 0.20	99.28 ± 0.29	2/2/4 of 473
MCNN	randsize-pad-constant	random	19.03 ± 0.00	3.20 ± 0.00	383/383/383 of 473
VarCNN	native	random	99.86 ± 0.10	99.90 ± 0.07	1/0/1 of 473
VarCNN	native-constant (no content)	random	97.04 ± 1.20	97.02 ± 1.23	22/10/10 of 473
VarCNN	randsize	random	99.86 ± 0.10	99.91 ± 0.06	1/1/0 of 473
VarCNN	randsize-constant	random	19.03 ± 0.00	3.20 ± 0.00	383/383/383 of 473
MCNN	pad (paper setup)	group	97.15 ± 1.45	97.71 ± 1.06	4/19/19 of 505
MCNN	randsize-pad	group	90.41 ± 2.29	88.84 ± 1.32	35/46/61 of 505
VarCNN	native	group	95.75 ± 1.28	95.94 ± 1.31	13/28/22 of 505
VarCNN	randsize	group	96.57 ± 0.36	95.68 ± 0.80	17/15/19 of 505
VarCNN	randsize	random, dedup t=0.9	99.72 ± 0.40	99.79 ± 0.30	0/0/2 of 235
MCNN	randsize-pad	random, dedup t=0.9	98.44 ± 0.72	98.59 ± 0.28	2/6/3 of 235
Random forest	(h, w) only	random	98.17 ± 0.70	98.36 ± 0.64	
Random forest	(h, w) only, 90/10	random	98.59 ± 1.11	98.70 ± 0.93	

5.1 Image size leaks the class

The probes isolate what the models can learn from crop size and shape alone. With padding (the paper's setup), MCNN reaches 98.31 ± 0.35% with no sonar content at all, close to the 98.17 ± 0.70% of a random forest on (h, w). Removing padding does not remove the cue: VarCNN on constant images of the original size still reaches 97.04 ± 1.20%, because convolutions see the image boundary. Resizing every image to a random size independent of its class removes it completely: both randsize probes fall to 19.03%, the share of the largest class (bottle). With real content, both models still score above 99% on a random split, so on random splits the size cue is available but not needed.

5.2 Near-duplicates inflate random splits

Table 7. Nearest training image for each test image, using 32x32 thumbnails (no size information, no learning). Mean over 3 splits.

Split	1-NN accuracy	Median best similarity	Best > 0.90	Best > 0.95
random	100.00 ± 0.00	0.927	71.2%	30.3%
block	100.00 ± 0.00	0.927	71.8%	30.9%
group	92.01 ± 0.94	0.760	0.0%	0.0%

On a random split, a nearest-neighbour lookup nearly matches the CNNs, because most test images have an almost identical frame in training (Figure 3). A split by file-number blocks behaves like a random split, since file numbers are shuffled. The group split removes every pair above 0.85 and is the only split that lowers accuracy (Figure 2). There, MCNN depends on the shortcuts: it loses 2.6 points with padding and 9 points once size is also removed, with 35 to 61 errors per seed. VarCNN loses 3 to 4 points and has the lowest spread of any model on the group split.

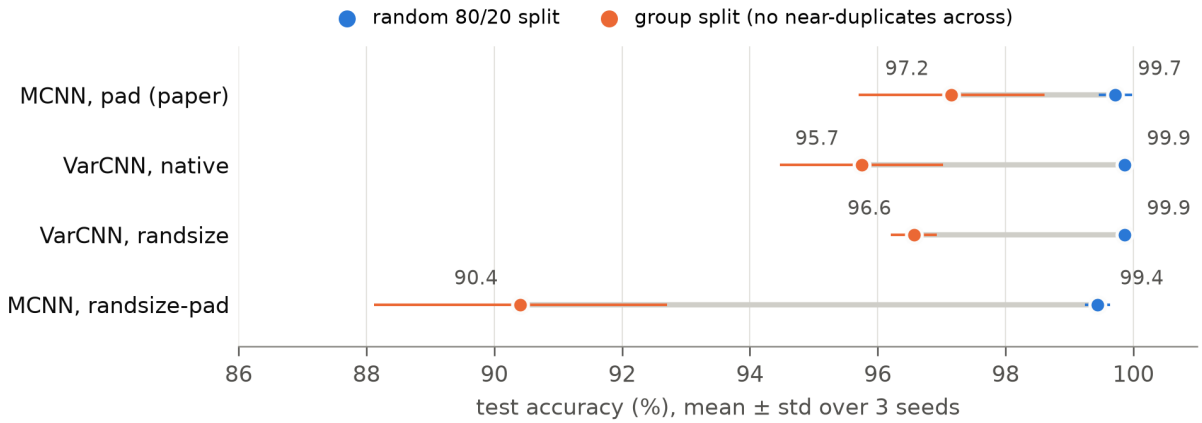


Figure 2. Accuracy on a random split vs the group split for the four setups run on both. Dots are means over 3 seeds, whiskers ± 1 std. Note the axis starts at 86%.

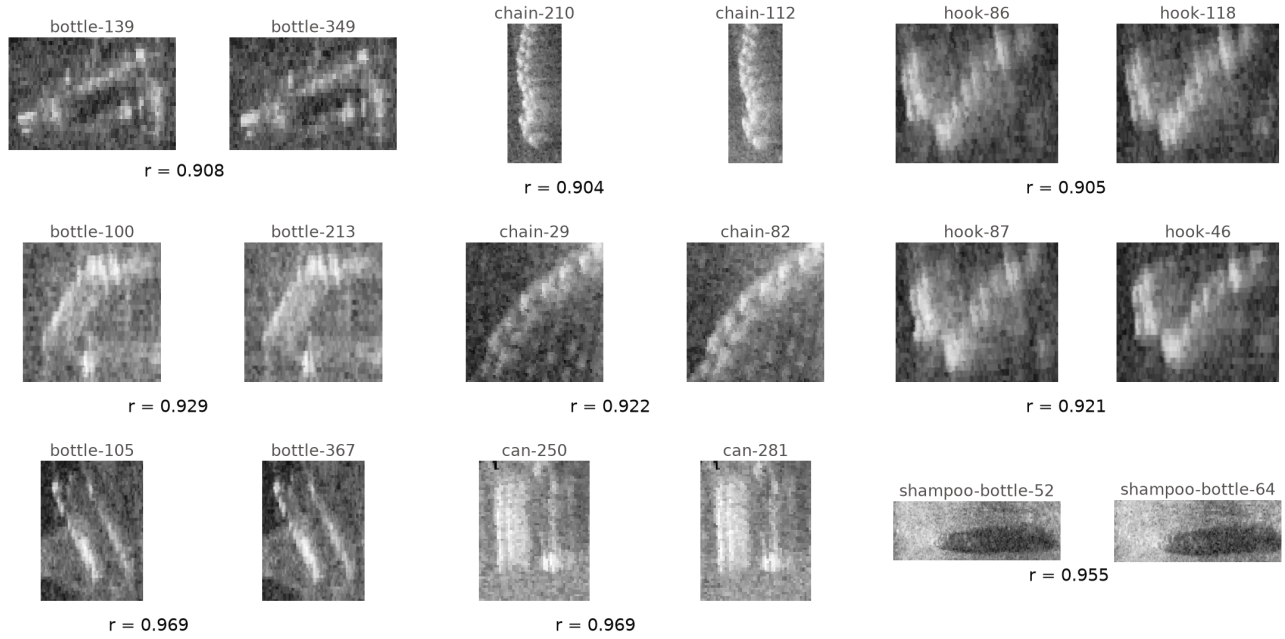


Figure 3. Removed near-duplicate pairs at three similarity levels (thumbnail correlation r). Even pairs just above 0.90 are the same object in the same pose, frames apart.

5.3 Deduplicated dataset

Within each class, images are visited in a seeded random order and kept only if their thumbnail correlation with every image already kept is below a threshold. At 0.9 this keeps 1,172 of 2,364 images (`data/dedup_t90/`, with a manifest recording what each removed image duplicates).

Table 8. Images kept per class at each threshold (carton = drink-carton, propel. = propeller, shamp./stand. = shampoo/standing bottle).

Threshold	Total	bottle	can	chain	carton	hook	propel.	shamp.	stand.	tire	valve
original	2364	449	367	226	349	133	137	99	65	331	208
0.95	1890	318	310	208	312	104	124	61	39	226	188
0.90	1172	218	196	162	164	57	76	36	23	124	116
0.85	750	138	121	118	89	33	54	21	18	84	74

On the 0.9 dataset with a random split, VarCNN (randsize) scores 99.72 ± 0.40% and MCNN (randsize-pad) 98.44 ± 0.72%, barely below the full dataset. Greedy deduplication at 0.9 leaves frames just under the threshold, which still come from the same stretch of a recording, so every test image keeps a close relative in training. It gives a smaller, less redundant dataset but not a leak-free evaluation; the group split remains the honest test.

6. Implications for the paper

- The headline 99.98% (abstract, Tables 3 and 5) is inconsistent with the 99.87% in the Results text and with Table 4. This replication gets 99.86 ± 0.20% at 90/10, supporting 99.87%.
- Table 3's standard deviations are impossible: with ~237 test images, one error in one of 10 runs already gives 99.96%, and a mean of 99.98% allows a standard deviation of at most ~0.06, not 0.58.
- Random image-level splits put near-duplicate frames on both sides. A split by recording sequence (or the group split here) is needed; accuracy then drops to about 97% for MCNN as published.
- Zero-padding hand-cropped images to 305x305 exposes the crop size, which alone predicts the class with about 98% accuracy. Any claim about learning from sonar returns needs a control for this.
- The 'fewer parameters' claim does not hold: 95% of MCNN's 2.8M parameters are in fc1, and a fully convolutional network with 185k parameters is more accurate once shortcuts are removed.
- Other issues from the review: Table 5 lists MobileNet at 96.7% while the text calls it the 99.7% previous best; Table 6 gives SqueezeNet ~112M parameters (it has ~1.2M); Table 5 compares against results on different data and splits; no optimizer, learning rate, epochs or validation set are reported.

7. Limitations and next steps

- The groups only approximate recording sequences. The full watertank release includes the original sequences and would allow a true split by sequence.
- 3 seeds per setting and no validation set; reported accuracy is at the final epoch with fixed hyperparameters, untuned for either model.
- No per-class metrics yet (the paper's Table 4 equivalent); this needs a rerun of the main settings.
- Random resizing distorts aspect ratio; it removes the size cue but also changes object shape. A scale-only augmentation with size jitter at test time would be a gentler alternative.

Appendix: reproducing

```
./run_all.sh   ./run_randsize.sh   ./run_group.sh
uv run python -m sonar_cnn.dupes
uv run python -m sonar_cnn.dedup --threshold 0.9   ./run_dedup.sh
uv run pytest
uv run --group report python report/build_report.py
```